



Міністерство освіти та науки України

Національний технічний університет України “Київський
політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи №2

з дисципліни “Розробка мобільних застосунків під Android”

Виконав:

Студент 3 курсу гр.

ПІ-33

Походун В.О.

Перевірів:

Орленко С. П.

“4” квітня 2026

Мета роботи: дослідити створення та взаємодію з компонентом Фрагмент (Fragment) компоненту Діяльність та набути практичні навички з використання фрагментів для інтерфейсу користувача.

ЗАВДАННЯ

Написати програму під платформу Андроїд, яка має інтерфейс, побудований з декількох фрагментів згідно варіанту. Перший фрагмент представляє з себе форму для введення даних та кнопку підтвердження («ОК»), а інший фрагмент відображає результат взаємодії. Тобто другий фрагмент містить тестове поле з результатом та кнопкою «Cancel» (якщо згідно варіанту така існує, якщо ж за варіантом її немає – можете додати за власним бажанням), яка очищає або приховує (або видаляє) другий фрагмент та очищає форму введення з першого фрагменту. Зверніть увагу, що робота з фрагментами відбувається в рамках однієї Діяльності.

Вікно вибору поїзду містить: два текстові поля (пункти відправлення та прибуття), групу опцій (час відправлення) та кнопку «ОК». Вивести інформацію щодо вибору при натисканні на кнопку «ОК» у деяке текстове поле.

Посилання на відео демонстрацію застосунку:

 Android Lab2 Recording.mkv

КОНТРОЛЬНІ ПИТАННЯ

1. Призначення та можливості компоненту Фрагмент.

Fragment — це модульна частина інтерфейсу всередині Activity, яка має власний життєвий цикл. Фрагменти дозволяють розбивати екран на незалежні блоки, які можна повторно використовувати в різних Activity. Це особливо

корисно для адаптації під різні розміри екранів — наприклад, на планшеті можна показати два фрагменти одночасно, а на телефоні — по одному. Також фрагменти спрощують навігацію та динамічну зміну вмісту екрану без створення нових Activity.

2. Життєвий цикл компонента Фрагмент.

Життєвий цикл фрагмента пов'язаний з життєвим циклом Activity, але має додаткові методи: `onAttach()` (прив'язка до Activity) -> `onCreate()` -> `onCreateView()` (створення UI) -> `onViewCreated()` -> `onStart()` -> `onResume()` -> `onPause()` -> `onStop()` -> `onDestroyView()` (знищення UI) -> `onDestroy()` -> `onDetach()` (від'єднання від Activity). Ключова відмінність від Activity — наявність `onCreateView/onDestroyView`, бо вигляд фрагмента може створюватися та знищуватися окремо від самого фрагмента.

3. Способи створення компонента Фрагмент.

Є два основні способи. Статичний — фрагмент оголошується прямо в XML-розмітці Activity через тег `<fragment>` або `<FragmentContainerView>`. Динамічний — фрагмент додається програмно через `FragmentManager` та `FragmentTransaction`, використовуючи методи `add()`, `replace()` тощо. Динамічний спосіб є більш гнучким, бо дозволяє змінювати фрагменти під час роботи застосунку.

4. Опишіть способи управління компонентом Фрагмент.

Управління фрагментами здійснюється через `FragmentManager` та `FragmentTransaction`. Основні операції: `add()` — додати фрагмент, `replace()` — замінити один фрагмент іншим, `remove()` — видалити фрагмент. Також можна використовувати `show()/hide()` для показу та приховування, та `addToBackStack()` для збереження транзакції у стеку навігації, щоб користувач міг повернутися назад кнопкою Back. Кожна транзакція завершується викликом `commit()`.

5. Опишіть способи взаємодії між Фрагментами.

Фрагменти не повинні взаємодіяти напряму. Основні способи: через спільну Activity (один фрагмент передає дані Activity, а та передає іншому фрагменту), через спільну ViewModel (обидва фрагменти спостерігають за одними й тими самими даними), та через Fragment Result API (setFragmentResult / setFragmentResultListener). Також можна використовувати інтерфейси зворотного виклику (callback interfaces).

6. Наведіть поняття системи, малої системи та мобільної платформи.

Система — це сукупність взаємопов'язаних елементів, які працюють разом для досягнення певної мети. Мала система — це система з обмеженими ресурсами (пам'ять, процесор, енергія), яка призначена для виконання конкретного набору задач, наприклад вбудовані системи або мобільні пристрої. Мобільна платформа — це програмно-апаратна основа для роботи мобільних застосунків, що включає операційну систему, набір API та інструменти розробки (наприклад, Android, iOS).

7. Опишіть типи мобільних застосунків.

Є три основні типи: нативні — розроблені під конкретну платформу (Kotlin/Java для Android, Swift для iOS), мають повний доступ до апаратних можливостей пристрою та найкращу продуктивність. Веб-застосунки — працюють у браузері, написані на HTML/CSS/JavaScript, кросплатформні, але з обмеженим доступом до функцій пристрою. Гібридні — поєднують обидва підходи, пишуться на веб-технологіях, але загортаються в нативну оболонку (наприклад, Ionic, Cordova), що дає доступ до API пристрою.

8. Наведіть класифікацію та загальну характеристику середовищ розробки мобільних застосунків.

Середовища розробки поділяються на: нативні IDE — Android Studio (для Android), Xcode (для iOS), які надають повний набір інструментів для

розробки під конкретну платформу. Кросплатформні фреймворки — Flutter (Dart), React Native (JavaScript), Kotlin Multiplatform — дозволяють писати код один раз і запускати на різних платформах. Гібридні середовища — Cordova, Ionic — використовують веб-технології для створення мобільних застосунків. Також є low-code/no-code платформи для швидкого створення простих застосунків без глибоких знань програмування.

9. Наведіть класифікацію та загальну характеристику мобільних платформ.

Основні мобільні платформи: Android — відкрита платформа від Google на базі ядра Linux, найбільша частка ринку, розробка на Kotlin/Java, розповсюдження через Google Play. iOS — закрита платформа від Apple, розробка на Swift/Objective-C, розповсюдження через App Store, відома високим рівнем безпеки та оптимізації. Також існували HarmonyOS (Huawei), KaiOS (для кнопочових телефонів) та вже застарілі Windows Phone і BlackBerry OS. Платформи класифікують за відкритістю (відкриті/закриті), архітектурою ядра та екосистемою розробки.